Numerical Programming

Final Exam Problem 2: Intercept a moving ball

Demetre Lataria

January 20, 2025

Chapter 1

It all begins with...

1.1 Problem Statement

The system aims to solve a real-time physics-based interception problem where a projectile (shot ball) must be launched to intercept a moving target ball while accounting for air resistance and gravity. The challenge involves:

- Real-time tracking of moving objects in video
- Estimation of physical parameters (mass, gravity)
- Prediction of target trajectory
- Calculation of optimal launch parameters for interception

Chapter 2

Look into abyss...

2.1 Mathematical Model

2.1.1 Physical Forces

The system models two primary forces acting on the balls:

1. Gravitational Force:

$$\vec{F_g} = m\vec{g} \tag{2.1}$$

2. Drag Force:

$$\vec{F_d} = -\frac{C_d}{m}|\vec{v}|\vec{v} \tag{2.2}$$

where:

- m is the mass of the ball
- $\vec{g}$ is the gravitational acceleration
- C_d is the drag coefficient
- $\vec{v}$ is the velocity vector

2.1.2 Equations of Motion

The system uses a second-order differential equation system:

$$\frac{d^2\vec{r}}{dt^2} = \vec{g} - \frac{C_d}{m}|\vec{v}|\vec{v} \tag{2.3}$$

where $\vec{r}$ is the position vector.

2.1.3 Boundary Conditions

Initial conditions for the shot ball:

$$\vec{r}(0) = \vec{r_0} \text{ (initial position)}$$
$$\vec{v}(0) = \vec{v_0} \text{ (initial velocity)}$$

Target condition:

$$\|\vec{r}_{shot}(t_{hit}) - \vec{r}_{target}(t_{hit})\| < \epsilon \tag{2.4}$$

where ϵ is the collision detection threshold.

Chapter 3

And the abyss stared back...

3.1 Numerical Methods

3.1.1 Integration Methods

The system implements two numerical integration schemes:

Euler Method

Basic implementation with adaptive step size:

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t$$
$$\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \Delta t$$

Runge-Kutta 4 (RK4)

More accurate integration with stages:

$$\begin{aligned} k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} k_1) \\ k_3 &= f(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} k_2) \\ k_4 &= f(t_n + \Delta t, y_n + \Delta t k_3) \\ y_{n+1} &= y_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

3.1.2 Error Analysis

The code implements adaptive step size control based on local error estimation:

$$\text{error} = \max(\|\vec{r}_1 - \vec{r}_2\|, \|\vec{v}_1 - \vec{v}_2\|) \tag{3.1}$$

where subscripts 1 and 2 represent single-step and double-step solutions respectively.
Step size adjustment:

$$\Delta t_{new} = \begin{cases} 0.5\Delta t & \text{if error} > \text{tolerance} \\ 2\Delta t & \text{if error} < 0.1 \times \text{tolerance} \end{cases} \tag{3.2}$$

Chapter 4

Cog in the Machine

4.1 Algorithm

4.1.1 Main Components

Algorithm 1 Ball Interception System

```
Initialize tracking system and physical parameters
while video is playing do
    Process frame to detect moving objects
    if time ¡ shoot_after_sec then
        Estimate target mass and gravity
        Predict target trajectory
    else if not shot then
        Calculate interception trajectory using shooting method
        Launch shot ball
    else
        Update shot ball position
        Check for collision
    end if
end while
```

4.1.2 Shooting Method Implementation

The shooting method iteratively adjusts initial velocity to achieve interception:

Algorithm 2 Shooting Method

```
Initialize guess for initial velocity
while not converged do
    Simulate trajectory with current velocity
    Calculate error from target
    if error ¡= tolerance then
        return current solution
    end if
    Adjust velocity based on error
end while
```

Chapter 5

Embrace the abyss

5.1 Implementation Analysis

5.1.1 Code Structure

The implementation is organized into several key classes:

- `WorldProperties`: Manages physical constants and parameters
- `Ball`: Implements ball physics and collision detection
- `Object`: Handles object tracking and velocity calculation
- `Tracker`: Implements object tracking algorithms
- `MotionDetector`: Coordinates video processing and simulation

5.1.2 Performance Considerations

Adaptive Step Size Control:

1. Balances accuracy and computational efficiency
2. Prevents numerical instability
3. Maintains real-time performance

Object Tracking Optimization:

1. Uses efficient region growing algorithm

2. Implements velocity-based prediction
3. Handles object disappearance and reappearance

5.1.3 Accuracy Metrics

The code tracks and reports:

- Position error in trajectory prediction
- Velocity error in numerical integration
- Target hit accuracy (If hit)
- Computational performance metrics

Chapter 6

The fruits of labor

6.1 Examples and Results

I am using my code from CP1 and CP2 projects. Object detection is little simplified and optimized for 2D ball detection. Videos were generated by $ball\_simple.py$ script. There are two directories: Videos and Results. Videos folder contains all the generated media and text document describing simulation parameters used for generation. In Results folder you will find results of the interception simulation with Euler and RK4 in seperate folders with simulatation video and text document containing accuarcy metrics and estimations. I will include both CP1 and CP2 if there are any further questions about my object detection or ball motion estimation code.

For background handling use the same steps as for Final task 1: disable $skip\_preprocessing$, use $cluster\_leak$, $cluster\_merge$.

6.1.1 ball_interceptee0

Simulation properties:

```
WORLD:
    C_d=0.004699999999999999
     gravity=245.25
     mass=1000
INITIAL CONDITIONS:
    position=[ 50. 150.]
    velocity=[200. 100.]
```

Estimated properties and results (Euler):

```
Initializing shooting method: [0 0], [534.13856716 597.74802403]
    real target position: [534.13856716 597.74802403],  estimated target position:[534.13758167 597.74806005]
    guessed initial velocity: [535.10885995 484.92139782],  end velocity when shot ball meets target:[533.09350768 711.26698556]
    Error:
        estimation method:
            position error: 5.551115123125783e-17,
            velocity error: 0.0,
        shooting method: 0.0009861452314199209

WORLD:
    C_d=0.004699999999999999
   esitmated gravity=224.07527652652382
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=1.4 sec
TARGET HIT:     True
```

Estimated properties and results (RK4):

```
Initializing shooting method: [0 0], [534.13490183 597.9230251 ]
    real target position: [534.13490183 597.9230251 ],  estimated target position:[534.13392863 597.92297453]
    guessed initial velocity: [535.10745465 484.86005497],  end velocity when shot ball meets target:[533.09179627 711.20666326]
    Error:
        estimation method:
            position error: 5.700995231450179e-14,
            velocity error: 1.2490009027033011e-14,
        shooting method: 0.0009745165542804147

WORLD:
    C_d=0.004699999999999999
   esitmated gravity=224.07527652652382
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=1.4 sec
TARGET HIT:     True
```

6.1.2 ball_interceptee1

Simulation properties:

```
WORLD:
    C_d=0.004699999999999999
     gravity=98.10000000000001
     mass=100
INITIAL CONDITIONS:
    position=[ 50. 150.]
    velocity=[200. 100.]
```

Estimated properties and results (Euler):

```
Initializing shooting method: [0 0], [625.05731086 294.70495236]
    real target position: [625.05731086 294.70495236],  estimated target position:[625.05631795 294.70497949]
    guessed initial velocity: [626.06063905 245.25971773],  end velocity when shot ball meets target:[624.02908439 344.41989755]
    Error:
        estimation method:
            position error: 5.551115123125783e-17,
            velocity error: 0.0,
        shooting method: 0.000993279073352306

Target hit error: 17.06191466655658
WORLD:
    C_d=0.004699999999999999
   esitmated gravity=98.20074707878808
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=2 sec
TARGET HIT:     True
```

Estimated properties and results (RK4):

```
Initializing shooting method: [0 0], [625.05501509 294.80972872]
    real target position: [625.05501509 294.80972872],  estimated target position:[625.05405424 294.80969163]
    guessed initial velocity: [626.06054732 245.26094324],  end velocity when shot ball meets target:[624.02887508 344.42152284]
    Error:
        estimation method:
            position error: 2.042810365310288e-14,
            velocity error: 1.609823385706477e-15,
        shooting method: 0.00096156790922208

Target hit error: 17.062147941086124
WORLD:
    C_d=0.004699999999999999
   esitmated gravity=98.20074707878808
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=2 sec
TARGET HIT:     True
```

6.1.3 ball_interceptee1 fail due to not enough time given for estimation

Simulation properties:

```
WORLD:
    C_d=0.004699999999999999
     gravity=98.10000000000001
     mass=100
INITIAL CONDITIONS:
    position=[ 50. 150.]
    velocity=[200. 100.]
```

Estimated properties and results (Euler):

```
Initializing shooting method: [0 0], [555.956339   154.71589642]
    real target position: [555.956339   154.71589642], estimated target position:[555.95536923 154.71584275]
    guessed initial velocity: [556.70446989 116.66407885], end velocity when shot ball meets target:[555.19554196 192.96339688]
    Error:
        estimation method:
            position error: 0.0,
            velocity error: 1.3877787807814457e-17,
        shooting method: 0.0009712502845617256

Target hit error: 53.86336562007637
WORLD:
    C_d=0.004699999999999999
   esitmated gravity=75.8788307987022
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=1.5 sec
TARGET HIT:     False
```

Estimated properties and results (RK4):

```
Initializing shooting method: [0 0], [555.95410083 154.7958309 ]
    real target position: [555.95410083 154.7958309 ], estimated target position:[555.9531143 154.7958083]
    guessed initial velocity: [556.7038067  116.66445223], end velocity when shot ball meets target:[555.19483462 192.9640016 ]
    Error:
        estimation method:
            position error: 1.4779844015322396e-14,
            velocity error: 7.216449660063518e-16,
        shooting method: 0.0009867909179953288

Target hit error: 53.862870106673576
WORLD:
    C_d=0.004699999999999999
   esitmated gravity=75.8788307987022
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=1.5 sec
TARGET HIT:     False
```

Explanation

Since my code is processing only positions with minimum distance of the threshold parameter and calculating acceleration using more spaced velocities for noise reduction purposes the algorithm requires more time and information to build accurate enough trajectory and intercept the ball. In this case the time given to the algorithm was not enough to gather enough information.

6.1.4 ball_interceptee2 shooting on ascention

Simulation properties:

```
WORLD:
    C_d=0.004699999999999999
     gravity=98.10000000000001
     mass=100
INITIAL CONDITIONS:
    position=[400. 600.]
    velocity=[  0. 300.]
```

Estimated properties and results (Euler):

```
Initializing shooting method: [0 0], [400.          148.04868914]
    real target position: [400.          148.04868914], estimated target position:[399.99902292 148.0486302 ]
    guessed initial velocity: [400.39479582  98.75036939], end velocity when shot ball meets target:[399.59136834 197.58982449]
    Error:
        estimation method:
            position error: 1.3877787807814457e-17,
            velocity error: 0.0,
        shooting method: 0.0009788606642216738

Target hit error: 1.034137268188357
WORLD:
    C_d=0.004699999999999999
   esitmated gravity=98.54079017582455
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=2 sec
TARGET HIT:     True
```

Estimated properties and results (RK4):

```
Initializing shooting method: [0 0], [400.          148.15149331]
    real target position: [400.          148.15149331], estimated target position:[399.99904089 148.1515166 ]
    guessed initial velocity: [400.39568084  98.75018601], end velocity when shot ball meets target:[399.5921865  197.58987335]
    Error:
        estimation method:
            position error: 3.4014457916953234e-14,
            velocity error: 4.066191827689636e-15,
        shooting method: 0.000959395688323421

Target hit error: 1.0349300140629285
WORLD:
    C_d=0.004699999999999999
   esitmated gravity=98.54079017582455
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=2 sec
TARGET HIT:     True
```

6.1.5 ball_interceptee2 fail due to decelleration and stopping of the target ball

Simulation properties:

```
WORLD:
    C_d=0.004699999999999999
     gravity=98.10000000000001
     mass=100
INITIAL CONDITIONS:
    position=[400. 600.]
    velocity=[  0. 300.]
```

Estimated properties and results (Euler):

```
Initializing shooting method: [0 0], [400.        169.983354]
    real target position: [400.        169.983354], estimated target position:[399.99903599 169.98331182]
    guessed initial velocity: [400.40143278 117.85304858], end velocity when shot ball meets target:[399.5826863  222.37236202]
    Error:
        estimation method:
            position error: 2.7755575615628914e-17,
            velocity error: 1.3877787807814457e-17,
        shooting method: 0.0009649295679697861

Target hit error: 26.79137161902881
WORLD:
    C_d=0.004699999999999999
   esitmated gravity=104.16973743130096
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=3 sec
TARGET HIT:     False
```

Estimated properties and results (RK4):

```
Initializing shooting method: [0 0], [400.         170.09202734]
    real target position: [400.         170.09202734], estimated target position:[399.99901059 170.09208818]
    guessed initial velocity: [400.40229649 117.85282705], end velocity when shot ball meets target:[399.58347056 222.37239788]
    Error:
        estimation method:
            position error: 3.556877015142845e-14,
            velocity error: 4.8433479449272454e-15,
        shooting method: 0.0009912796949006508

Target hit error: 26.79161881742444
WORLD:
    C_d=0.004699999999999999
   esitmated gravity=104.16973743130096
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=3 sec
TARGET HIT:     False
```

Explanation

Again as stated in previous failure example, because of the way algorithm estimates velocity and acceleration it doesn't work well with ball that stops and plans to accelerate later. This could have been mitigated by using history of velocities and accelerations but this introduces problem with responsiveness of the algorithm to change in a timely matter. However this doesn't mean that the algorithm can't handle the accelerating and then decellerating target which we observe successfuly intercepted in previous tests.

6.1.6 ball_interceptee2 shooting on descention

Simulation properties:

```
WORLD:
    C_d=0.004699999999999999
     gravity=98.10000000000001
     mass=100
INITIAL CONDITIONS:
    position=[400. 600.]
    velocity=[  0. 300.]
```

Estimated properties and results (Euler):

```
Initializing shooting method: [0 0], [400.         335.9599376]
    real target position: [400.         335.9599376], estimated target position:[399.99901512 335.95996077]
    guessed initial velocity: [400.4795086  284.98966074], end velocity when shot ball meets target:[399.49654938 387.20217241]
    Error:
        estimation method:
            position error: 2.7755575615628914e-17,
            velocity error: 2.7755575615628914e-17,
        shooting method: 0.000985150839687048

Target hit error: 6.852858690063412
WORLD:
    C_d=0.004699999999999999
   esitmated gravity=101.38430418057733
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=4 sec
TARGET HIT:     True
```

Estimated properties and results (RK4):

```
Initializing shooting method: [0 0], [400.         336.06718413]
    real target position: [400.         336.06718413], estimated target position:[399.99902186 336.06719663]
    guessed initial velocity: [400.48060197 284.9902463 ], end velocity when shot ball meets target:[399.49751341 387.2031307 ]
    Error:
        estimation method:
            position error: 1.3378187446733136e-14,
            velocity error: 2.1371793224034263e-15,
        shooting method: 0.0009782155827237388

Target hit error: 6.852429833671348
WORLD:
    C_d=0.004699999999999999
   esitmated gravity=101.38430418057733
   estimated_mass=100
SHOOTING_METHOD:
   shot_after=4 sec
TARGET HIT:     True
```

Chapter 7

The end

7.1 Conclusion

Algorithm works with great success and with modifications to the velocity and acceleration calculation we get nice and clean data points with stable and reasonably accurate values. The algorithm isn't perfect and may not intercept the ball all of the time as seen in uppermentioned test cases. Floating point precision and arithmetic issues can influence a trajectory approximation. However it handles approximation well with reasonable parameter values.